

In [123...

```
import numpy as np
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objs as go
import plotly.express as px
import calmap
import folium
import plotly.figure_factory as ff
from plotly.offline import init_notebook_mode, iplot
import plotly.graph_objs as go
import plotly.offline as py
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot
active_case = '#002bd8'
confirmed_cases = '#af0000'
death_cases = '#1d0826'
recovered_cases = '#8fce00'
from pandas.plotting import register_matplotlib_converters
register_matplotlib_converters()
import warnings
```

In [124...

```
pip install notebook
```

Requirement already satisfied: notebook in c:\users\urmil\anaconda3\lib\site-packages (6.5.4)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: jinja2 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (3.1.2)  
Requirement already satisfied: tornado>=6.1 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (6.3.2)  
Requirement already satisfied: pyzmq>=17 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (23.2.0)  
Requirement already satisfied: argon2-cffi in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (21.3.0)  
Requirement already satisfied: traitlets>=4.2.1 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (5.7.1)  
Requirement already satisfied: jupyter-core>=4.6.1 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (5.3.0)  
Requirement already satisfied: jupyter-client>=5.3.4 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (7.4.9)  
Requirement already satisfied: ipython-genutils in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (0.2.0)  
Requirement already satisfied: nbformat in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (5.9.2)  
Requirement already satisfied: nbconvert>=5 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (6.5.4)  
Requirement already satisfied: nest-asyncio>=1.5 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (1.5.6)  
Requirement already satisfied: ipykernel in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (6.25.0)  
Requirement already satisfied: Send2Trash>=1.8.0 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (1.8.0)  
Requirement already satisfied: terminado>=0.8.3 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (0.17.1)  
Requirement already satisfied: prometheus-client in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (0.14.1)  
Requirement already satisfied: nbclassic>=0.4.7 in c:\users\urmil\anaconda3\lib\site-packages (from notebook) (0.5.5)  
Requirement already satisfied: entrypoints in c:\users\urmil\anaconda3\lib\site-packages (from jupyter-client>=5.3.4->notebook) (0.4)  
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\urmil\anaconda3\lib\site-packages (from jupyter-client>=5.3.4->notebook) (2.8.2)  
Requirement already satisfied: platformdirs>=2.5 in c:\users\urmil\anaconda3\lib\site-packages (from jupyter-core>=4.6.1->notebook) (3.10.0)  
Requirement already satisfied: pywin32>=300 in c:\users\urmil\anaconda3\lib\site-packages (from jupyter-core>=4.6.1->notebook) (305.1)  
Requirement already satisfied: jupyter-server>=1.8 in c:\users\urmil\anaconda3\lib\site-packages (from nbclassic>=0.4.7->notebook) (1.23.4)  
Requirement already satisfied: notebook-shim>=0.1.0 in c:\users\urmil\anaconda3\lib\site-packages (from nbclassic>=0.4.7->notebook) (0.2.2)  
Requirement already satisfied: lxml in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (4.9.3)  
Requirement already satisfied: beautifulsoup4 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (4.12.2)  
Requirement already satisfied: bleach in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (4.1.0)  
Requirement already satisfied: defusedxml in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (0.7.1)  
Requirement already satisfied: jupyterlab-pygments in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (0.1.2)  
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (2.1.1)  
Requirement already satisfied: mistune<2,>=0.8.1 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (0.8.4)  
Requirement already satisfied: nbclient>=0.5.0 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (0.5.13)

Requirement already satisfied: packaging in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (23.1)

Requirement already satisfied: pandocfilters>=1.4.1 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (1.5.0)

Requirement already satisfied: pygments>=2.4.1 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (2.15.1)

Requirement already satisfied: tinycss2 in c:\users\urmil\anaconda3\lib\site-packages (from nbconvert>=5->notebook) (1.2.1)

Requirement already satisfied: fastjsonschema in c:\users\urmil\anaconda3\lib\site-packages (from nbformat->notebook) (2.16.2)

Requirement already satisfied: jsonschema>=2.6 in c:\users\urmil\anaconda3\lib\site-packages (from nbformat->notebook) (4.17.3)

Requirement already satisfied: pywinpty>=1.1.0 in c:\users\urmil\anaconda3\lib\site-packages (from terminado>=0.8.3->notebook) (2.0.10)

Requirement already satisfied: argon2-cffi-bindings in c:\users\urmil\anaconda3\lib\site-packages (from argon2-cffi->notebook) (21.2.0)

Requirement already satisfied: comm>=0.1.1 in c:\users\urmil\anaconda3\lib\site-packages (from ipykernel->notebook) (0.1.2)

Requirement already satisfied: debugpy>=1.6.5 in c:\users\urmil\anaconda3\lib\site-packages (from ipykernel->notebook) (1.6.7)

Requirement already satisfied: ipython>=7.23.1 in c:\users\urmil\anaconda3\lib\site-packages (from ipykernel->notebook) (8.15.0)

Requirement already satisfied: matplotlib-inline>=0.1 in c:\users\urmil\anaconda3\lib\site-packages (from ipykernel->notebook) (0.1.6)

Requirement already satisfied: psutil in c:\users\urmil\anaconda3\lib\site-packages (from ipykernel->notebook) (5.9.0)

Requirement already satisfied: backcall in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (0.2.0)

Requirement already satisfied: decorator in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (5.1.1)

Requirement already satisfied: jedi>=0.16 in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (0.18.1)

Requirement already satisfied: pickleshare in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (0.7.5)

Requirement already satisfied: prompt-toolkit!=3.0.37,<3.1.0,>=3.0.30 in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (3.0.36)

Requirement already satisfied: stack-data in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (0.2.0)

Requirement already satisfied: colorama in c:\users\urmil\anaconda3\lib\site-packages (from ipython>=7.23.1->ipykernel->notebook) (0.4.6)

Requirement already satisfied: attrs>=17.4.0 in c:\users\urmil\anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat->notebook) (22.1.0)

Requirement already satisfied: pyparsing!=0.17.0,!0.17.1,!0.17.2,>=0.14.0 in c:\users\urmil\anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat->notebook) (0.18.0)

Requirement already satisfied: anyio<4,>=3.1.0 in c:\users\urmil\anaconda3\lib\site-packages (from jupyter-server>=1.8->nbclassic>=0.4.7->notebook) (3.5.0)

Requirement already satisfied: websocket-client in c:\users\urmil\anaconda3\lib\site-packages (from jupyter-server>=1.8->nbclassic>=0.4.7->notebook) (0.58.0)

Requirement already satisfied: six>=1.5 in c:\users\urmil\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->jupyter-client>=5.3.4->notebook) (1.16.0)

Requirement already satisfied: cffi>=1.0.1 in c:\users\urmil\anaconda3\lib\site-packages (from argon2-cffi-bindings->argon2-cffi->notebook) (1.15.1)

```

Requirement already satisfied: soupsieve>1.2 in c:\users\urmil\anaconda3\lib\site-packages (from beautifulsoup4->nbconvert>=5->notebook) (2.4)
Requirement already satisfied: webencodings in c:\users\urmil\anaconda3\lib\site-packages (from bleach->nbconvert>=5->notebook) (0.5.1)
Requirement already satisfied: idna>=2.8 in c:\users\urmil\anaconda3\lib\site-packages (from anyio<4,>=3.1.0->jupyter-server>=1.8->nbclassic>=0.4.7->notebook) (3.4)
Requirement already satisfied: sniffio>=1.1 in c:\users\urmil\anaconda3\lib\site-packages (from anyio<4,>=3.1.0->jupyter-server>=1.8->nbclassic>=0.4.7->notebook) (1.2.0)
Requirement already satisfied: pycparser in c:\users\urmil\anaconda3\lib\site-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->notebook) (2.21)
Requirement already satisfied: parso<0.9.0,>=0.8.0 in c:\users\urmil\anaconda3\lib\site-packages (from jedi>=0.16->ipython>=7.23.1->ipykernel->notebook) (0.8.3)
Requirement already satisfied: wcwidth in c:\users\urmil\anaconda3\lib\site-packages (from prompt-toolkit!=3.0.37,<3.1.0,>=3.0.30->ipython>=7.23.1->ipykernel->notebook) (0.2.5)
Requirement already satisfied: executing in c:\users\urmil\anaconda3\lib\site-packages (from stack-data->ipython>=7.23.1->ipykernel->notebook) (0.8.3)
Requirement already satisfied: asttokens in c:\users\urmil\anaconda3\lib\site-packages (from stack-data->ipython>=7.23.1->ipykernel->notebook) (2.0.5)
Requirement already satisfied: pure-eval in c:\users\urmil\anaconda3\lib\site-packages (from stack-data->ipython>=7.23.1->ipykernel->notebook) (0.2.2)

```

In [125...

```
!pip install folium
```

```

Requirement already satisfied: folium in c:\users\urmil\anaconda3\lib\site-packages (0.15.1)
Requirement already satisfied: branca>=0.6.0 in c:\users\urmil\anaconda3\lib\site-packages (from folium) (0.7.0)
Requirement already satisfied: Jinja2>=2.9 in c:\users\urmil\anaconda3\lib\site-packages (from folium) (3.1.2)
Requirement already satisfied: numpy in c:\users\urmil\anaconda3\lib\site-packages (from folium) (1.24.3)
Requirement already satisfied: requests in c:\users\urmil\anaconda3\lib\site-packages (from folium) (2.31.0)
Requirement already satisfied: xyzservices in c:\users\urmil\anaconda3\lib\site-packages (from folium) (2022.9.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\urmil\anaconda3\lib\site-packages (from Jinja2>=2.9->folium) (2.1.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (1.26.16)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (2023.11.17)

```

In [126...

```
!pip install --upgrade folium
```

```
Requirement already satisfied: folium in c:\users\urmil\anaconda3\lib\site-packages (0.15.1)
Requirement already satisfied: branca>=0.6.0 in c:\users\urmil\anaconda3\lib\site-packages (from folium) (0.7.0)
Requirement already satisfied: jinja2>=2.9 in c:\users\urmil\anaconda3\lib\site-packages (from folium) (3.1.2)
Requirement already satisfied: numpy in c:\users\urmil\anaconda3\lib\site-packages (from folium) (1.24.3)
Requirement already satisfied: requests in c:\users\urmil\anaconda3\lib\site-packages (from folium) (2.31.0)
Requirement already satisfied: xyzservices in c:\users\urmil\anaconda3\lib\site-packages (from folium) (2022.9.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\urmil\anaconda3\lib\site-packages (from jinja2>=2.9->folium) (2.1.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (1.26.1)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (2023.11.17)
```

In [127... `!pip install calmap`

```
Requirement already satisfied: calmap in c:\users\urmil\anaconda3\lib\site-packages (0.0.11)
Requirement already satisfied: matplotlib in c:\users\urmil\anaconda3\lib\site-packages (from calmap) (3.7.2)
Requirement already satisfied: numpy in c:\users\urmil\anaconda3\lib\site-packages (from calmap) (1.24.3)
Requirement already satisfied: pandas in c:\users\urmil\anaconda3\lib\site-packages (from calmap) (2.0.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (1.0.5)
Requirement already satisfied: cycler>=0.10 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (4.25.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (23.1)
Requirement already satisfied: pillow>=6.2.0 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (10.0.1)
Requirement already satisfied: pyparsing<3.1,>=2.3.1 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\urmil\anaconda3\lib\site-packages (from pandas->calmap) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in c:\users\urmil\anaconda3\lib\site-packages (from pandas->calmap) (2023.3)
Requirement already satisfied: six>=1.5 in c:\users\urmil\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->calmap) (1.16.0)
```

In [128... `import pandas as pd`

```
confirmed_df = pd.read_csv("C:\\Users\\urmil\\Downloads\\Capstone Project Data Set Avengers Team\\time_series_covid_19_confirmed.
recovered_df = pd.read_csv("C:\\Users\\urmil\\Downloads\\Capstone Project Data Set Avengers Team\\time_series_covid_19_recovered.
deaths_df = pd.read_csv("C:\\Users\\urmil\\Downloads\\Capstone Project Data Set Avengers Team\\time_series_covid_19_deaths.csv")
```

In [129... confirmed\_df.head()

Out[129]:

|   | Province/State | Country/Region | Lat       | Long      | 1/22/2020 | 1/23/2020 | 1/24/2020 | 1/25/2020 | 1/26/2020 | 1/27/2020 | ... | 11/21/2020 | 11/22/2020 |
|---|----------------|----------------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----|------------|------------|
| 0 | NaN            | Afghanistan    | 33.93911  | 67.709953 | 0         | 0         | 0         | 0         | 0         | 0         | ... | 44503      | 44         |
| 1 | NaN            | Albania        | 41.15330  | 20.168300 | 0         | 0         | 0         | 0         | 0         | 0         | ... | 32196      | 32         |
| 2 | NaN            | Algeria        | 28.03390  | 1.659600  | 0         | 0         | 0         | 0         | 0         | 0         | ... | 73774      | 74         |
| 3 | NaN            | Andorra        | 42.50630  | 1.521800  | 0         | 0         | 0         | 0         | 0         | 0         | ... | 6207       | 6          |
| 4 | NaN            | Angola         | -11.20270 | 17.873900 | 0         | 0         | 0         | 0         | 0         | 0         | ... | 14413      | 14         |

5 rows × 318 columns

In [130...  
dates = confirmed\_df.columns[4:]  
confirmed\_df\_long = confirmed\_df.melt(id\_vars = ['Province/State', 'Country/Region', 'Lat', 'Long'], value\_vars = dates, var\_name = 'Date')  
deaths\_df\_long = deaths\_df.melt(id\_vars = ['Province/State', 'Country/Region', 'Lat', 'Long'], value\_vars = dates, var\_name = 'Date')  
recovered\_df\_long = recovered\_df.melt(id\_vars = ['Province/State', 'Country/Region', 'Lat', 'Long'], value\_vars = dates, var\_name = 'Date')  
complete\_table = pd.concat([confirmed\_df\_long, deaths\_df\_long['Deaths'], recovered\_df\_long['Recovered']], axis = 1, sort = False)  
complete\_table.tail()

Out[130]:

|       | Province/State | Country/Region     | Lat        | Long       | Date       | Confirmed | Deaths | Recovered |
|-------|----------------|--------------------|------------|------------|------------|-----------|--------|-----------|
| 86659 | NaN            | Vietnam            | 14.058324  | 108.277199 | 11/30/2020 | 1347      | 35     | NaN       |
| 86660 | NaN            | West Bank and Gaza | 31.952200  | 35.233200  | 11/30/2020 | 85647     | 732    | NaN       |
| 86661 | NaN            | Yemen              | 15.552727  | 48.516388  | 11/30/2020 | 2077      | 605    | NaN       |
| 86662 | NaN            | Zambia             | -13.133897 | 27.849332  | 11/30/2020 | 17647     | 357    | NaN       |
| 86663 | NaN            | Zimbabwe           | -19.015438 | 29.154857  | 11/30/2020 | 9950      | 276    | NaN       |

In [131...  
complete\_table.to\_csv("C:\\Users\\urmil\\Downloads\\Capstone Project Data Set Avengers Team\\covid\_19\_clean\_data1.csv", index=False)  
complete\_table = pd.read\_csv("C:\\Users\\urmil\\Downloads\\Capstone Project Data Set Avengers Team\\covid\_19\_clean\_data1.csv", parse\_dates=True)

In [132... complete\_table.tail()

Out[132]:

|              | Province/State | Country/Region     | Lat        | Long       | Date       | Confirmed | Deaths | Recovered |
|--------------|----------------|--------------------|------------|------------|------------|-----------|--------|-----------|
| <b>86659</b> | NaN            | Vietnam            | 14.058324  | 108.277199 | 2020-11-30 | 1347      | 35     | NaN       |
| <b>86660</b> | NaN            | West Bank and Gaza | 31.952200  | 35.233200  | 2020-11-30 | 85647     | 732    | NaN       |
| <b>86661</b> | NaN            | Yemen              | 15.552727  | 48.516388  | 2020-11-30 | 2077      | 605    | NaN       |
| <b>86662</b> | NaN            | Zambia             | -13.133897 | 27.849332  | 2020-11-30 | 17647     | 357    | NaN       |
| <b>86663</b> | NaN            | Zimbabwe           | -19.015438 | 29.154857  | 2020-11-30 | 9950      | 276    | NaN       |

In [133...]

```
complete_table['Check'] = np.where(complete_table['Confirmed'] >= complete_table['Recovered'],complete_table['Confirmed'],np.nan)
complete_table[complete_table['Check'].isna()]
```

Out[133]:

|              | Province/State   | Country/Region     | Lat        | Long       | Date       | Confirmed | Deaths | Recovered | Check |
|--------------|------------------|--------------------|------------|------------|------------|-----------|--------|-----------|-------|
| <b>56</b>    | NaN              | Chad               | 15.454200  | 18.732200  | 2020-01-22 | 0         | 0      | 28.0      | NaN   |
| <b>230</b>   | NaN              | Slovenia           | 46.151200  | 14.995500  | 2020-01-22 | 0         | 0      | 2.0       | NaN   |
| <b>309</b>   | NaN              | Burkina Faso       | 12.238300  | -1.561600  | 2020-01-23 | 0         | 0      | 2.0       | NaN   |
| <b>317</b>   | Diamond Princess | Canada             | 0.000000   | 0.000000   | 2020-01-23 | 0         | 0      | 28.0      | NaN   |
| <b>491</b>   | NaN              | Russia             | 61.524010  | 105.318756 | 2020-01-23 | 0         | 0      | 2.0       | NaN   |
| ...          | ...              | ...                | ...        | ...        | ...        | ...       | ...    | ...       | ...   |
| <b>86659</b> | NaN              | Vietnam            | 14.058324  | 108.277199 | 2020-11-30 | 1347      | 35     | NaN       | NaN   |
| <b>86660</b> | NaN              | West Bank and Gaza | 31.952200  | 35.233200  | 2020-11-30 | 85647     | 732    | NaN       | NaN   |
| <b>86661</b> | NaN              | Yemen              | 15.552727  | 48.516388  | 2020-11-30 | 2077      | 605    | NaN       | NaN   |
| <b>86662</b> | NaN              | Zambia             | -13.133897 | 27.849332  | 2020-11-30 | 17647     | 357    | NaN       | NaN   |
| <b>86663</b> | NaN              | Zimbabwe           | -19.015438 | 29.154857  | 2020-11-30 | 9950      | 276    | NaN       | NaN   |

37004 rows × 9 columns

In [134...]

```
complete_table['Check'] = complete_table['Check'].fillna(0)
```

In [135...]

```
complete_table[complete_table['Check'].isna()]
```

Out[135]:

| Province/State | Country/Region | Lat | Long | Date | Confirmed | Deaths | Recovered | Check |
|----------------|----------------|-----|------|------|-----------|--------|-----------|-------|
|----------------|----------------|-----|------|------|-----------|--------|-----------|-------|

```
In [136... non_empty_rows = complete_table[complete_table['Province/State'].notna()]
print(non_empty_rows)
```

|       | Province/State                               | Country/Region | \ |
|-------|----------------------------------------------|----------------|---|
| 8     | Australian Capital Territory                 | Australia      |   |
| 9     | New South Wales                              | Australia      |   |
| 10    | Northern Territory                           | Australia      |   |
| 11    | Queensland                                   | Australia      |   |
| 12    | South Australia                              | Australia      |   |
| ...   | ...                                          | ...            |   |
| 86649 | Gibraltar                                    | United Kingdom |   |
| 86650 | Isle of Man                                  | United Kingdom |   |
| 86651 | Montserrat                                   | United Kingdom |   |
| 86652 | Saint Helena, Ascension and Tristan da Cunha | United Kingdom |   |
| 86653 | Turks and Caicos Islands                     | United Kingdom |   |

|       | Lat        | Long       | Date       | Confirmed | Deaths | Recovered | Check |
|-------|------------|------------|------------|-----------|--------|-----------|-------|
| 8     | -35.473500 | 149.012400 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 9     | -33.868800 | 151.209300 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 10    | -12.463400 | 130.845600 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 11    | -27.469800 | 153.025100 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 12    | -34.928500 | 138.600700 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| ...   | ...        | ...        | ...        | ...       | ...    | ...       | ...   |
| 86649 | 36.140800  | -5.353600  | 2020-11-30 | 1020      | 5      | NaN       | 0.0   |
| 86650 | 54.236100  | -4.548100  | 2020-11-30 | 369       | 25     | NaN       | 0.0   |
| 86651 | 16.742498  | -62.187366 | 2020-11-30 | 13        | 1      | NaN       | 0.0   |
| 86652 | -7.946700  | -14.355900 | 2020-11-30 | 3         | 0      | NaN       | 0.0   |
| 86653 | 21.694000  | -71.797900 | 2020-11-30 | 748       | 6      | NaN       | 0.0   |

[27004 rows x 9 columns]

```
In [137... pip install folium --upgrade
```



Requirement already satisfied: folium in c:\users\urmil\anaconda3\lib\site-packages (0.15.1)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: branca>=0.6.0 in c:\users\urmil\anaconda3\lib\site-packages (from folium) (0.7.0)  
Requirement already satisfied: Jinja2>=2.9 in c:\users\urmil\anaconda3\lib\site-packages (from folium) (3.1.2)  
Requirement already satisfied: numpy in c:\users\urmil\anaconda3\lib\site-packages (from folium) (1.24.3)  
Requirement already satisfied: requests in c:\users\urmil\anaconda3\lib\site-packages (from folium) (2.31.0)  
Requirement already satisfied: xyzservices in c:\users\urmil\anaconda3\lib\site-packages (from folium) (2022.9.0)  
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\urmil\anaconda3\lib\site-packages (from Jinja2>=2.9->folium) (2.1.1)  
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (2.0.4)  
Requirement already satisfied: idna<4,>=2.5 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (3.4)  
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (1.26.16)  
Requirement already satisfied: certifi>=2017.4.17 in c:\users\urmil\anaconda3\lib\site-packages (from requests->folium) (2023.11.17)

In [138...

```
import folium
from folium import plugins

mapp = folium.Map(
    location=[0, 0],
    tiles='cartodbpositron',
    min_zoom=1,
    max_zoom=4,
    zoom_start=1,
    attr='Map tiles by Carto, under CC BY 3.0. Data by OpenStreetMap, under ODbL.'
)

# Add a title to the map
title = '<h3 align="center" style="font-size:20px"><b>COVID-19 Cases by State</b></h3>'
mapp.get_root().html.add_child(folium.Element(title))

# Create a MarkerCluster for better visualization of markers
marker_cluster = folium.plugins.MarkerCluster().add_to(mapp)

# Iterate through the dataframe to add markers
for i in range(len(non_empty_rows)):
    # Check if the location values are not NaN
    if not pd.isna(non_empty_rows.iloc[i]['Lat']) and not pd.isna(non_empty_rows.iloc[i]['Long']):
        # Create a CircleMarker for each state
        folium.CircleMarker(
            location=[non_empty_rows.iloc[i]['Lat'], non_empty_rows.iloc[i]['Long']],
            color='crimson',
```

```
fill=True,  
fill_color='crimson',  
fill_opacity=0.6,  
tooltip=f'<strong>State: </strong>{non_empty_rows.iloc[i]["Province/State"]}' +  
        f'<br><strong>Confirmed: </strong>{non_empty_rows.iloc[i]["Confirmed"]}' +  
        f'<br><strong>Deaths: </strong>{non_empty_rows.iloc[i]["Deaths"]}' +  
        f'<br><strong>Recovered: </strong>{non_empty_rows.iloc[i]["Recovered"]}',  
radius=8  
) .add_to(marker_cluster)
```

```
# Display the map  
mapp
```

Out[138]: Make this Notebook Trusted to load map: File -> Trust Notebook

## COVID-19 Cases by State



In [139... non\_empty\_rows.tail(15)

Out[139]:

|       | Province/State                               | Country/Region | Lat        | Long       | Date       | Confirmed | Deaths | Recovered | Check |
|-------|----------------------------------------------|----------------|------------|------------|------------|-----------|--------|-----------|-------|
| 86581 | Aruba                                        | Netherlands    | 12.521100  | -69.968300 | 2020-11-30 | 4845      | 45     | NaN       | 0.0   |
| 86582 | Bonaire, Sint Eustatius and Saba             | Netherlands    | 12.178400  | -68.238500 | 2020-11-30 | 162       | 3      | NaN       | 0.0   |
| 86583 | Curacao                                      | Netherlands    | 12.169600  | -68.990000 | 2020-11-30 | 2379      | 4      | NaN       | 0.0   |
| 86584 | Sint Maarten                                 | Netherlands    | 18.042500  | -63.054800 | 2020-11-30 | 1066      | 25     | NaN       | 0.0   |
| 86643 | Anguilla                                     | United Kingdom | 18.220600  | -63.068600 | 2020-11-30 | 6         | 0      | NaN       | 0.0   |
| 86644 | Bermuda                                      | United Kingdom | 32.307800  | -64.750500 | 2020-11-30 | 260       | 9      | NaN       | 0.0   |
| 86645 | British Virgin Islands                       | United Kingdom | 18.420700  | -64.640000 | 2020-11-30 | 71        | 1      | NaN       | 0.0   |
| 86646 | Cayman Islands                               | United Kingdom | 19.313300  | -81.254600 | 2020-11-30 | 274       | 2      | NaN       | 0.0   |
| 86647 | Channel Islands                              | United Kingdom | 49.372300  | -2.364400  | 2020-11-30 | 1299      | 48     | NaN       | 0.0   |
| 86648 | Falkland Islands (Malvinas)                  | United Kingdom | -51.796300 | -59.523600 | 2020-11-30 | 16        | 0      | NaN       | 0.0   |
| 86649 | Gibraltar                                    | United Kingdom | 36.140800  | -5.353600  | 2020-11-30 | 1020      | 5      | NaN       | 0.0   |
| 86650 | Isle of Man                                  | United Kingdom | 54.236100  | -4.548100  | 2020-11-30 | 369       | 25     | NaN       | 0.0   |
| 86651 | Montserrat                                   | United Kingdom | 16.742498  | -62.187366 | 2020-11-30 | 13        | 1      | NaN       | 0.0   |
| 86652 | Saint Helena, Ascension and Tristan da Cunha | United Kingdom | -7.946700  | -14.355900 | 2020-11-30 | 3         | 0      | NaN       | 0.0   |
| 86653 | Turks and Caicos Islands                     | United Kingdom | 21.694000  | -71.797900 | 2020-11-30 | 748       | 6      | NaN       | 0.0   |

In [140...]

```
# Assuming populations
population_bermuda = 64000
population_Aruba = 106277

# Select rows for Bermuda and Queensland
bermuda_data = complete_table[complete_table['Province/State'] == 'Bermuda']
Aruba_data = complete_table[complete_table['Province/State'] == 'Aruba']

# Get the latest data
latest_bermuda = bermuda_data.iloc[-1]
latest_Aruba = Aruba_data.iloc[-1]

# Calculate confirmation rates
confirmation_rate_bermuda = (latest_bermuda['Confirmed'] / population_bermuda) * 100
```

```
confirmation_rate_Aruba = (latest_Aruba ['Confirmed'] / population_Aruba) * 100

print(f"Confirmation rate for Bermuda: {confirmation_rate_bermuda:.2f}%")
print(f"Confirmation rate for Aruba : {confirmation_rate_Aruba:.2f}%")
```

Confirmation rate for Bermuda: 0.41%  
Confirmation rate for Aruba : 4.56%

In [141...

```
# Assuming populations
population_bermuda = 64000
population_Aruba = 106277

# Select rows for Bermuda and Queensland
bermuda_data = complete_table[complete_table['Province/State'] == 'Bermuda']
Aruba_data = complete_table[complete_table['Province/State'] == 'Aruba']

# Get the Latest data
latest_bermuda = bermuda_data.iloc[-1]
latest_Aruba = Aruba_data.iloc[-1]

# Calculate confirmation rates
Death_rate_bermuda = (latest_bermuda['Deaths'] / population_bermuda) * 100
Death_rate_Aruba = (latest_Aruba['Deaths'] / population_Aruba) * 100

print(f"Death rate for Bermuda: {Death_rate_bermuda:.2f}%")
print(f"Death rate for Aruba: {Death_rate_Aruba:.2f}%")
```

Death rate for Bermuda: 0.01%  
Death rate for Aruba: 0.04%

In [142...

```
from scipy.stats import pearsonr

# Assuming you have the rates calculated previously
confirmation_rates = np.array([confirmation_rate_bermuda, confirmation_rate_Aruba])
death_rates = np.array([Death_rate_bermuda, Death_rate_Aruba])

# Calculate correlation coefficients
correlation_confirmation_death, _ = pearsonr(confirmation_rates, death_rates)

print(f"Correlation between Confirmation Rate and Death Rate: {correlation_confirmation_death:.2f}")
```

Correlation between Confirmation Rate and Death Rate: 1.00

In [143]...

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# Data
regions = ['Bermuda', 'Aruba']
confirmation_rates = [confirmation_rate_bermuda, confirmation_rate_Aruba]
death_rates = [Death_rate_bermuda, Death_rate_Aruba]

# Set up subplots
fig, axs = plt.subplots(2, 1, figsize=(8, 6))
fig.subplots_adjust(hspace=0.5)

# Create progress bar visualization for Confirmation Rates
confirmationBars = axs[0].barh(regions, [0, 0], color='blue', alpha=0.7)
axs[0].set_title('Confirmation Rates Progress Visualization')
axs[0].set_xlim(0, 1)

# Create progress bar visualization for Death Rates
deathBars = axs[1].barh(regions, [0, 0], color='red', alpha=0.7)
axs[1].set_title('Death Rates Progress Visualization')
axs[1].set_xlim(0, 1)

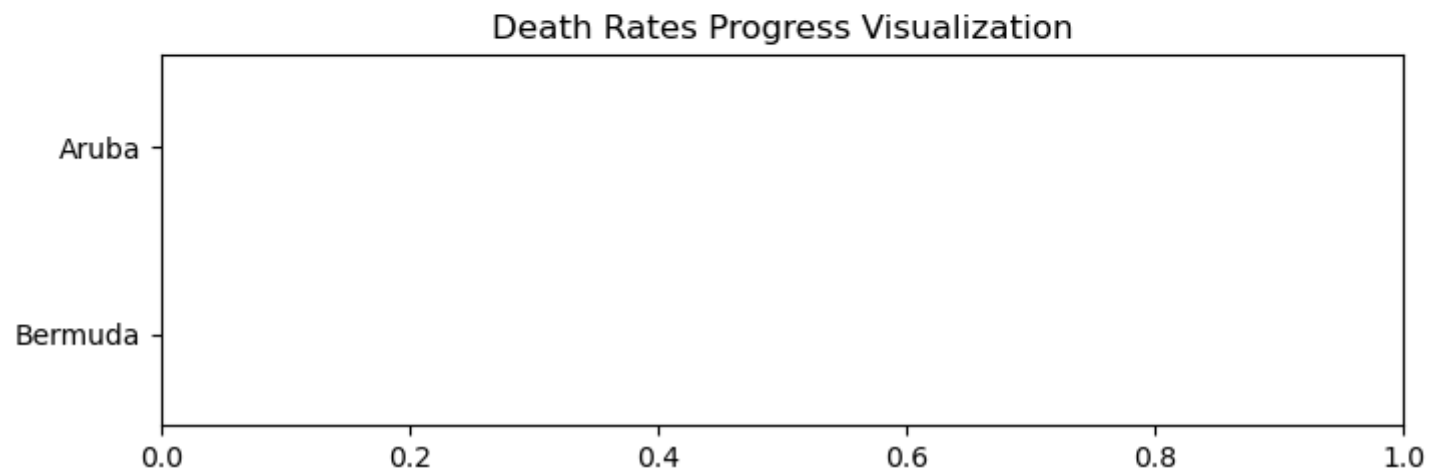
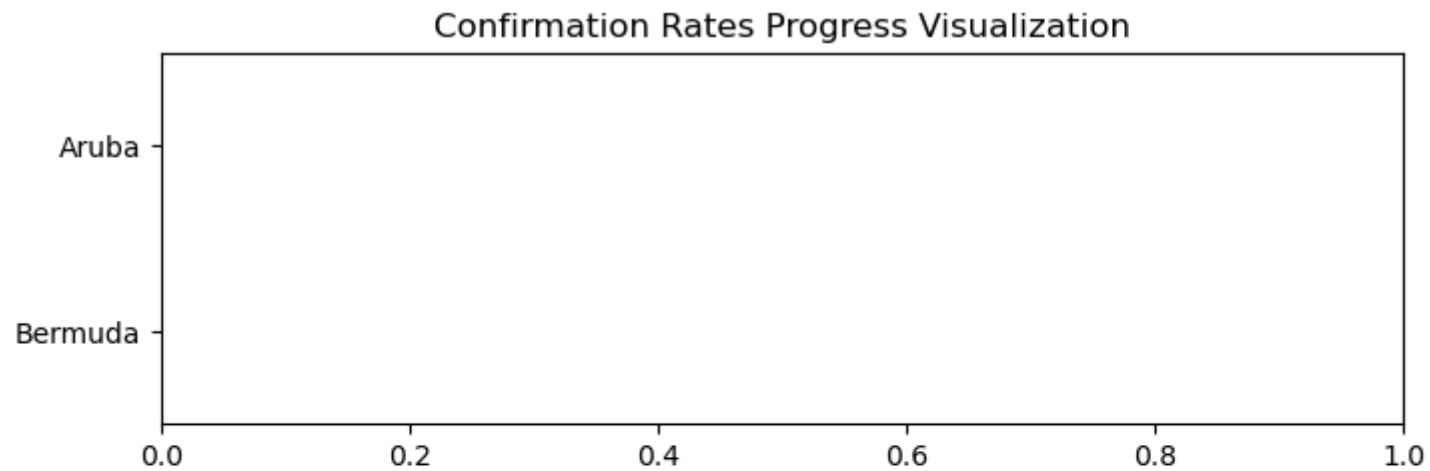
# Animation update function
def update(frame):
    for i in range(len(regions)):
        confirmationBars[i].set_width(confirmation_rates[i] * frame / frames)
        deathBars[i].set_width(death_rates[i] * frame / frames)

# Animate the progress bars
frames = 100 # Adjust the number of frames as needed
animation = FuncAnimation(fig, update, frames=frames, interval=50)

# Assuming you have defined html_file_path earlier in your code
html_file_path = 'path/to/your/file.html'

# Assuming you have already created the animation and saved it to an HTML file
HTML(html_file_path)
```

Out[143]: path/to/your/file.html



In [144...

```
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# Assuming you already have a figure 'fig' and the update function 'update' defined

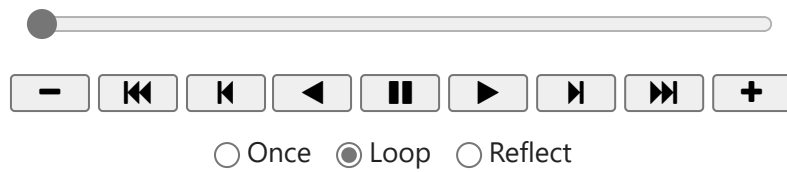
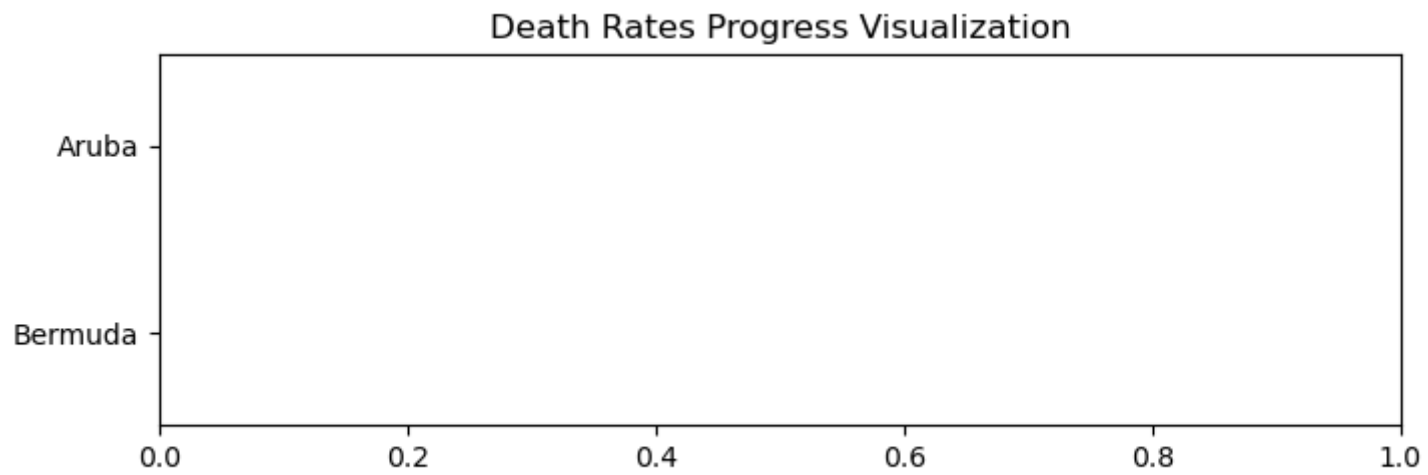
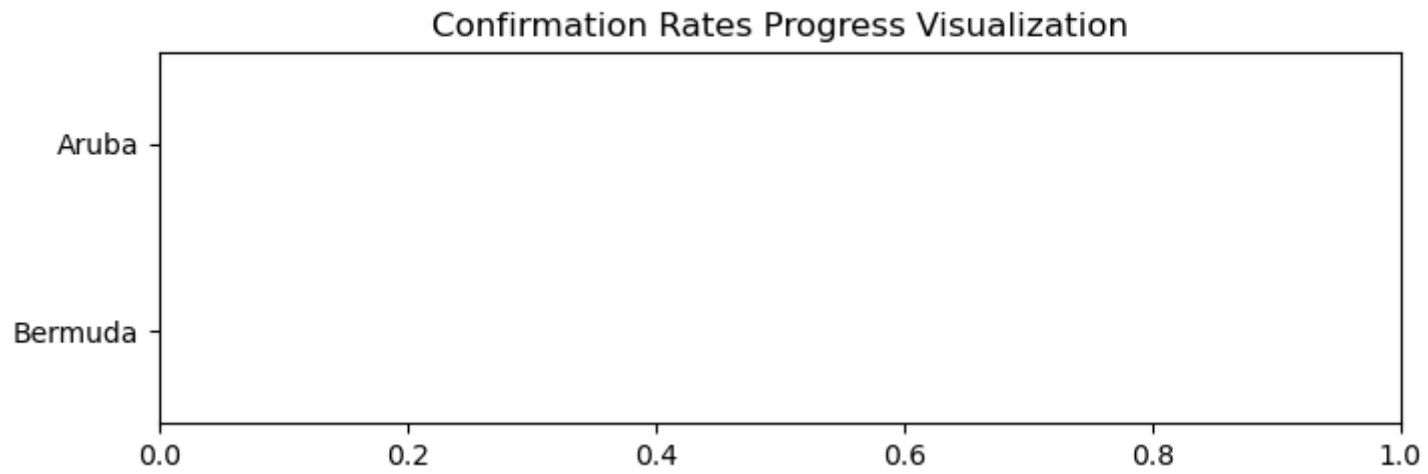
frames = 100 # Adjust the number of frames as needed
animation = FuncAnimation(fig, update, frames=frames, interval=50)

# Save the animation as a GIF file
animation.save('animated_plot.gif', writer='pillow', fps=30) # Change the file name as needed
```

```
# Display the animated plot  
HTML(animation.to_jshtml())
```



Out[144]:



In [145... `complete_table.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 86664 entries, 0 to 86663
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Province/State  27004 non-null  object
1   Country/Region  86664 non-null  object
2   Lat             86036 non-null  float64
3   Long            86036 non-null  float64
4   Date            86664 non-null  datetime64[ns]
5   Confirmed       86664 non-null  int64
6   Deaths         86664 non-null  int64
7   Recovered       81954 non-null  float64
8   Check          86664 non-null  float64
dtypes: datetime64[ns](1), float64(4), int64(2), object(2)
memory usage: 6.0+ MB
```

In [146... `#DROPPING UNNAMED COLUMN`  
`complete_table.drop(['Unnamed: 0'], axis=1, inplace=True, errors='ignore')`  
`complete_table.head()`

Out[146]:

|   | Province/State | Country/Region | Lat       | Long      | Date       | Confirmed | Deaths | Recovered | Check |
|---|----------------|----------------|-----------|-----------|------------|-----------|--------|-----------|-------|
| 0 | NaN            | Afghanistan    | 33.93911  | 67.709953 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 1 | NaN            | Albania        | 41.15330  | 20.168300 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 2 | NaN            | Algeria        | 28.03390  | 1.659600  | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 3 | NaN            | Andorra        | 42.50630  | 1.521800  | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |
| 4 | NaN            | Angola         | -11.20270 | 17.873900 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   |

In [147... `cases_list = ['Confirmed', 'Deaths', 'Recovered', 'Active']`  
`complete_table['Active'] = complete_table['Confirmed'] - complete_table['Deaths'] - complete_table['Recovered']`  
`complete_table['Country/Region'] = complete_table['Country/Region'].replace('Mainland China', 'China')`  
`complete_table['Country/Region'].fillna('', inplace=True)`  
`complete_table[cases_list].fillna(0, inplace=True)`  
  
`print("\nDataFrame After fillna:")`  
`print(complete_table.tail())`

DataFrame After fillna:

|       | Province/State | Country/Region     | Lat        | Long       | Date       | \ |
|-------|----------------|--------------------|------------|------------|------------|---|
| 86659 | NaN            | Vietnam            | 14.058324  | 108.277199 | 2020-11-30 |   |
| 86660 | NaN            | West Bank and Gaza | 31.952200  | 35.233200  | 2020-11-30 |   |
| 86661 | NaN            | Yemen              | 15.552727  | 48.516388  | 2020-11-30 |   |
| 86662 | NaN            | Zambia             | -13.133897 | 27.849332  | 2020-11-30 |   |
| 86663 | NaN            | Zimbabwe           | -19.015438 | 29.154857  | 2020-11-30 |   |

|       | Confirmed | Deaths | Recovered | Check | Active |
|-------|-----------|--------|-----------|-------|--------|
| 86659 | 1347      | 35     | NaN       | 0.0   | NaN    |
| 86660 | 85647     | 732    | NaN       | 0.0   | NaN    |
| 86661 | 2077      | 605    | NaN       | 0.0   | NaN    |
| 86662 | 17647     | 357    | NaN       | 0.0   | NaN    |
| 86663 | 9950      | 276    | NaN       | 0.0   | NaN    |

```
In [148... complete_table.drop(complete_table[complete_table['Active']< 0].index, axis = 0, inplace = True)
```

```
In [149... filter_substrings = lambda df: df[df['Province/State'].str.contains('Grand Princess') | df['Country/Region'].str.contains('Cruise')]
filter_by_country = lambda df, country: df[df['Country/Region'] == country]
filter_by_non_country = lambda df, country: df[df['Country/Region'] != country]
filter_by_latest_date = lambda df: df[df['Date'] == df['Date'].max()].reset_index()

s = filter_substrings(complete_table)
c = filter_by_country(complete_table, 'China')
r = filter_by_non_country(complete_table, 'China')
f_l = filter_by_latest_date(complete_table)
c_l = filter_by_country(f_l, 'China')
r_l = filter_by_non_country(f_l, 'China')
```

```
In [150... group_and_sum = lambda df, group_col: df.groupby(group_col)[['Confirmed', 'Deaths', 'Recovered', 'Active']].sum().reset_index()

f_l_g = group_and_sum(f_l, 'Country/Region')
c_l_g = group_and_sum(c_l, 'Province/State')
r_l_g = group_and_sum(r_l, 'Country/Region')
```

```
In [151... max(complete_table['Date'])
```

```
Out[151]: Timestamp('2020-11-30 00:00:00')
```

```
In [152... t_1 = complete_table.groupby(['Country/Region', 'Province/State'])[['Confirmed', 'Deaths', 'Recovered', 'Active']].max()
```

```
In [153... t_1 = complete_table.groupby(['Country/Region', 'Province/State'])[['Confirmed', 'Deaths', 'Recovered', 'Active']].max()
t = complete_table.groupby('Date')[['Confirmed', 'Deaths', 'Recovered', 'Active']].sum().reset_index()
t_latest = t[t['Date'] == max(t['Date'])].reset_index(drop=True)
styled_table = t_latest.style.background_gradient(cmap='Pastel2')
styled_table
```

```
Out[153]:
```

|   | Date                | Confirmed | Deaths  | Recovered | Active   |
|---|---------------------|-----------|---------|-----------|----------|
| 0 | 2020-11-30 00:00:00 | 63410725  | 1471623 | 0.000000  | 0.000000 |

```
In [154... melted_df = t.melt(id_vars="Date", value_vars=['Recovered', 'Active', 'Deaths'])

# Create a sunburst chart
fig = px.sunburst(
    melted_df,
    path=["variable", "Date"],
    values="value",
    title="Sunburst Chart of COVID-19 Cases Over Time",
    color_discrete_sequence=['#2ecc71', '#3498db', '#e74c3c'],
    height=500,
    width=800
)

fig.show()
```

In [155...

```
import plotly.express as px
t_f = f_l_g.sort_values(by='Confirmed', ascending=False).reset_index(drop=True)
t_f['Confirmed'] = t_f['Confirmed'].replace(0, 1e-6)

fig = px.treemap(t_f, path=['Country/Region'], values='Confirmed',
                 title='Total Confirmed Cases by Country/Region',
                 color='Confirmed', color_continuous_scale='Viridis')

fig.update_layout(
    autosize=False,
    width=800,
    height=600,
```

```
margin=dict(l=0, r=0, b=0, t=40),  
)  
fig.show()
```

```
In [156... non_empty_states = complete_table[complete_table['Province/State'].notna()]  
fig = px.line(non_empty_states, x='Date', y='Confirmed', color='Province/State',
```

```
        title='Confirmed Cases Over Time by State',
        labels={'Confirmed': 'Confirmed Cases', 'Date': 'Date'},
        markers=True)
fig.update_layout(
    autosize=False,
    width=1000,
    height=600,
    margin=dict(l=0, r=0, b=0, t=40),
)
fig.show()
```

In [157...

```
import plotly.express as px
t_f = f_l_g.sort_values(by='Active', ascending=False).reset_index(drop=True)
t_f['Active'] = t_f['Active'].replace(0, 1e-6) # Handle zero values
fig = px.treemap(t_f, path=['Country/Region'], values='Active',
```



```
        title='Total Active Cases by Country/Region',  
        color='Active', color_continuous_scale='Greens')  
fig.update_layout(  
    autosize=False,  
    width=800,  
    height=600,  
    margin=dict(l=0, r=0, b=0, t=40),  
)  
  
fig.show()
```

In [158...

```
import plotly.express as px

# Filter out rows with missing state information
non_empty_states = complete_table[complete_table['Province/State'].notna()]
```

```
# Create a Line chart for active cases over time, with different colors for each state
fig_active = px.line(non_empty_states, x='Date', y='Active', color='Province/State',
                    title='Active Cases Over Time by State',
                    labels={'Active': 'Active Cases', 'Date': 'Date'},
                    markers=True)

# Update the layout for better visibility
fig_active.update_layout(
    autosize=False,
    width=1000,
    height=600,
    margin=dict(l=0, r=0, b=0, t=40),
)

# Show the Line chart for active cases
fig_active.show()
```

◀  ▶

In [159...

```
import plotly.express as px
```

```
# Assuming 'complete_table' contains the necessary columns, including 'Date'  
fig_animated_deaths_map = px.choropleth(complete_table,
```

```
locations="Country/Region",
locationmode="country names",
color="Deaths",
hover_name="Country/Region",
range_color=[1, 1000], # Adjust the range as needed
color_continuous_scale="Reds",
animation_frame="Date",
title='Animated Deaths Over Time by Country/Region')

# Update the layout for better visibility
fig_animated_deaths_map.update_layout(
    autosize=False,
    width=800,
    height=600,
    margin=dict(l=0, r=0, b=0, t=40),
)

# Show the animated world map for deaths
fig_animated_deaths_map.show()
```

In [160...

```
import plotly.express as px
```

[illegible]

```
locationmode="country names",
color="Active",
hover_name="Country/Region",
range_color=[1, 5000], # Adjust the range as needed
color_continuous_scale="Blues",
animation_frame="Date",
title='Animated Active Cases Over Time by Country/Region')

# Update the layout for better visibility
fig_animated_active_map.update_layout(
    autosize=False,
    width=800,
    height=600,
    margin=dict(l=0, r=0, b=0, t=40),
)

# Show the animated world map for active cases
fig_animated_active_map.show()
```

In [161...

```
import plotly.express as px

# Assuming 'complete_table' is your DataFrame
complete_table['size'] = complete_table['Confirmed'].pow(0.3)
```



```
fig = px.scatter_polar(complete_table,  
                       r='size',  
                       theta='Country/Region',  
                       color="Confirmed",  
                       size="size",  
                       animation_frame="Date",  
                       title='Nightingale Rose Chart of Confirmed Cases Spread Over Time',  
                       color_continuous_scale='Viridis')  
  
fig.show()
```

In [162...

```
import plotly.express as px

# Assuming you already have f_gdf dataframe
complete_table['size'] = complete_table['Confirmed'].pow(0.3)

fig = px.scatter_geo(complete_table,
                     locations="Country/Region",
                     locationmode="country names",
                     color="Confirmed",
                     size='size',
                     hover_name="Country/Region",
                     range_color=[0, max(complete_table['Confirmed']) + 2],
                     projection="natural earth",
                     animation_frame="Date",
                     title='Confirmed cases spread over time')

fig.update_layout(coloraxis_showscale=False) # Fix layout update here
fig.show()
```

```
In [163... # Save the animation as a video  
fig.write_html("covid_pic2.html", auto_play=True)
```

```
In [164... f_gdf = complete_table.groupby(['Date', 'Country/Region'])[['Confirmed', 'Deaths', 'Recovered']].max()  
f_gdf = f_gdf.reset_index()  
f_gdf['Date'] = pd.to_datetime(f_gdf['Date'])  
f_gdf['Date'] = f_gdf['Date'].dt.strftime('%m%d%Y')  
f_gdf['size'] = f_gdf['Deaths'].pow(0.3)
```

```
In [165... fig = px.scatter_geo(f_gdf, locations="Country/Region", locationmode="country names", color="Deaths", size='size', hover_name="Country")
fig.update(layout_coloraxis_showscale=False)
fig.show()
```

```
In [166... f_gdf = complete_table.groupby(['Date', 'Country/Region'])[['Confirmed', 'Deaths', 'Recovered']].max()
f_gdf = f_gdf.reset_index()
f_gdf['Date'] = pd.to_datetime(f_gdf['Date'])
f_gdf['Date'] = f_gdf['Date'].dt.strftime('%m%d%Y')
f_gdf['size'] = f_gdf['Confirmed'].pow(0.3)
```

```
In [167... f_gdf = complete_table.groupby(['Date', 'Country/Region'])[['Confirmed', 'Deaths', 'Recovered', 'Active']].max()
f_gdf = f_gdf.reset_index()
f_gdf['Date'] = pd.to_datetime(f_gdf['Date'])
f_gdf['Date'] = f_gdf['Date'].dt.strftime('%m%d%Y')
f_gdf['size'] = f_gdf['Active'].pow(0.3)
f_gdf['Confirmed'].sort_values()
```

```
Out[167]: 0          0
4832       0
4831       0
4829       0
4828       0
...
40915    13005807
41108    13213995
41301    13369528
41494    13509762
41687    13670332
Name: Confirmed, Length: 41701, dtype: int64
```

```
In [168... import plotly.express as px

# Assuming 'complete_table' is your DataFrame
# Perform the groupby and sum operation
t = complete_table.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered']].sum().reset_index()

# Create a stacked area chart
fig = px.area(t, x='Date', y='Confirmed', color='Country/Region',
              title='Stacked Area Chart of COVID-19 Confirmed Cases Across Nations',
              color_discrete_sequence=px.colors.qualitative.Set1)

# Show the figure
fig.show()
```

In [169...

```
import plotly.express as px

# Assuming 'complete_table' is your DataFrame
# Perform the groupby and sum operation
t = complete_table.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered']].sum().reset_index()

# Create a stacked area chart for death cases
fig = px.area(t, x='Date', y='Deaths', color='Country/Region',
              title='Stacked Area Chart of COVID-19 Death Cases Across Nations',
              color_discrete_sequence=px.colors.qualitative.Set1)
```

```
# Show the figure  
fig.show()
```

In [170...

```
import plotly.express as px  
  
# Assuming 'complete_table' is your DataFrame  
# Perform the groupby and sum operation  
t = complete_table.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered']].sum().reset_index()
```

```
# Create a stacked area chart for recovered cases
fig = px.area(t, x='Date', y='Recovered', color='Country/Region',
              title='Stacked Area Chart of COVID-19 Recovered Cases Across Nations',
              color_discrete_sequence=px.colors.qualitative.Set1)

# Show the figure
fig.show()
```

In [171... `import plotly.express as px`



```
# Assuming 'complete_table' is your DataFrame
# Perform the groupby and sum operation
t = complete_table.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered', 'Active']].sum().reset_index()

# Create a stacked area chart for active cases
fig = px.area(t, x='Date', y='Active', color='Country/Region',
              title='Stacked Area Chart of COVID-19 Active Cases Across Nations',
              color_discrete_sequence=px.colors.qualitative.Set1)

# Show the figure
fig.show()
```

In [172...

```
import plotly.express as px

# Assuming 'complete_table' is your DataFrame
t = complete_table.groupby('Date').sum().reset_index()
t['Number of Deaths to 1000 Confirmed cases'] = round(t['Deaths'] / t['Confirmed'], 3) * 1000
t['Number of Recovered to 1000 Confirmed cases'] = round(t['Recovered'] / t['Confirmed'], 3) * 1000

# Melt the DataFrame for Gantt chart
t = t.melt(id_vars='Date', value_vars=['Number of Deaths to 1000 Confirmed cases', 'Number of Recovered to 1000 Confirmed cases'],
          var_name='Ratio', value_name='Value')
```

```
# Create Gantt chart
fig = px.bar(t, x='Date', y='Value', color='Ratio',
             title='Mortality and Recovery Rates Over Time',
             labels={'Value': 'Rate per 1000 Confirmed Cases'},
             color_discrete_sequence=['#e74c3c', '#2ecc71'])

# Show the figure
fig.show()
```

In [173...

```
import plotly.express as px

# Assuming 'complete_table' is your DataFrame
cc = complete_table.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered']].max().reset_index()

# China Visualization
t_china = cc[cc['Country/Region'] == 'China'].reset_index()
fig_china = px.line(t_china, x='Date', y=['Confirmed', 'Deaths', 'Recovered'],
                    color_discrete_sequence=['#1f77b4', '#d62728', '#2ca02c'],
                    title='Parallel Coordinates Plot - China')

# Rest of the Countries Visualization
t_rest = cc[cc['Country/Region'] != 'China'].groupby('Date').sum().reset_index()
fig_rest = px.line(t_rest, x='Date', y=['Confirmed', 'Deaths', 'Recovered'],
                  color_discrete_sequence=['#1f77b4', '#d62728', '#2ca02c'],
                  title='Parallel Coordinates Plot - Rest of the Countries')

# Show the figures
fig_china.show()
fig_rest.show()
```



In [174...

```
import plotly.graph_objects as go
import pandas as pd

# Assuming 'complete_table' is your DataFrame
cc = complete_table.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered', 'Active']].max().reset_index()

# Function to convert date to numerical values for radar chart
def convert_date_to_numeric(df):
    df['Date'] = pd.to_datetime(df['Date'])
    df['NumericDate'] = (df['Date'] - df['Date'].min()) / pd.Timedelta(days=1)
```

```

    return df

# China Visualization
t_china = cc[cc['Country/Region'] == 'China'].reset_index()
t_china = convert_date_to_numeric(t_china)

# US Visualization
t_us = cc[cc['Country/Region'] == 'US'].groupby('Date').sum().reset_index()
t_us = convert_date_to_numeric(t_us)

# Radar Chart Function
def radar_chart(df, title, color):
    fig = go.Figure()

    for i, row in df.iterrows():
        fig.add_trace(go.Scatterpolar(
            r=row[['Confirmed', 'Deaths', 'Recovered', 'Active']],
            theta=['Confirmed', 'Deaths', 'Recovered', 'Active'],
            fill='toself',
            name=str(row['NumericDate']),
            line_color=color
        ))

    fig.update_layout(
        polar=dict(
            radialaxis=dict(
                visible=True,
            ),
        ),
        showlegend=True,
        title=title
    )

    return fig

# Create Radar Charts
fig_china = radar_chart(t_china, 'Radar Chart - China', confirmed_cases)
fig_us = radar_chart(t_us, 'Radar Chart - US', confirmed_cases)

# Show the figures
fig_china.show()
fig_us.show()

```





In [175...

```
import plotly.express as px

def china_ornot(row):
    if row['Country/Region'] == 'China':
        return 'From China'
    else:
        return 'Outside China'

t = complete_table.copy()
t['Country/Region'] = t.apply(china_ornot, axis=1)
```

```

t = t.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered', 'Active']]
t = t.sum().diff().reset_index()
m = t['Country/Region'] != t['Country/Region'].shift(1)
t.loc[m, 'Confirmed'] = np.nan
t.loc[m, 'Deaths'] = np.nan
t.loc[m, 'Recovered'] = np.nan
t.loc[m, 'Active'] = np.nan

# Function to create a grouped bar plot
def create_bar_plot(data, y_col, color_col, title, textangle=None):
    fig = px.bar(data, x="Date", y=y_col, color=color_col, barmode="group",
                 text=y_col, title=title, color_discrete_sequence=[confirmed_cases, death_cases, recovered_cases, active_case])

    fig.update_traces(textposition='outside', textangle=textangle)
    fig.show()

# Create grouped bar plots for each case type
create_bar_plot(t, 'Confirmed', 'Country/Region', 'Confirmed Cases', textangle=-90)
create_bar_plot(t, 'Deaths', 'Country/Region', 'Death Cases', textangle=-90)
create_bar_plot(t, 'Recovered', 'Country/Region', 'Recovered Cases', textangle=-90)
create_bar_plot(t, 'Active', 'Country/Region', 'Active Cases', textangle=-90)

```







In [176...

```
def loc(row):  
    if row['Country/Region']=='China':  
        if row['Province/State'] == 'Hubei':  
            return 'Hubei'  
        else:  
            return 'Other Chinese Provinces'  
    else:  
        return 'Rest of the world'  
  
t = complete_table.copy()
```

```
t['Region'] = t.apply(loc, axis=1)
t['Date'] = t['Date'].dt.strftime('%Y-%m-%d')
#t = t.groupby(['Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered', 'Active']].sum().reset_index()
t = t.melt(id_vars=['Region', 'Date'], value_vars=['Confirmed', 'Deaths', 'Recovered', 'Active'],
          var_name='Case', value_name='Count').sort_values(['Date', 'Case'])

t.head()
fig = px.bar(t, y="Region", x="Count", color='Case', barmode="group", orientation='h', text="Count", title="Hubei - China - World", anim
fig.update_traces(textposition='outside')
fig.show()
```

In [177...

```
import plotly.express as px

def china_ornot(row):
    if row['Country/Region'] == 'China':
        return 'From China'
    else:
        return 'Outside China'

t = complete_table.copy()
t['Country/Region'] = t.apply(china_ornot, axis=1)
t = t.groupby(['Country/Region', 'Date'])[['Confirmed', 'Deaths', 'Recovered', 'Active']]
t = t.sum().diff().reset_index()
m = t['Country/Region'] != t['Country/Region'].shift(1)
t.loc[m, 'Confirmed'] = np.nan
t.loc[m, 'Deaths'] = np.nan
t.loc[m, 'Recovered'] = np.nan
t.loc[m, 'Active'] = np.nan

# Function to create a strip plot
def create_strip_plot(data, y_col, color_col, title):
    fig = px.strip(data, x="Country/Region", y=y_col, color=color_col,
                  title=title, color_discrete_sequence=[confirmed_cases, death_cases, recovered_cases, active_case])

    fig.show()

# Create strip plots for each case type
create_strip_plot(t, 'Confirmed', 'Country/Region', 'Strip Plot - Confirmed Cases Over Time')
create_strip_plot(t, 'Deaths', 'Country/Region', 'Strip Plot - Death Cases Over Time')
create_strip_plot(t, 'Recovered', 'Country/Region', 'Strip Plot - Recovered Cases Over Time')
create_strip_plot(t, 'Active', 'Country/Region', 'Strip Plot - Active Cases Over Time')
```









In [178... `!pip install calmap`

Requirement already satisfied: calmap in c:\users\urmil\anaconda3\lib\site-packages (0.0.11)  
 Requirement already satisfied: matplotlib in c:\users\urmil\anaconda3\lib\site-packages (from calmap) (3.7.2)  
 Requirement already satisfied: numpy in c:\users\urmil\anaconda3\lib\site-packages (from calmap) (1.24.3)  
 Requirement already satisfied: pandas in c:\users\urmil\anaconda3\lib\site-packages (from calmap) (2.0.3)  
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (1.0.5)  
 Requirement already satisfied: cycler>=0.10 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (0.11.0)  
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (4.25.0)  
 Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (1.4.4)  
 Requirement already satisfied: packaging>=20.0 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (23.1)  
 Requirement already satisfied: pillow>=6.2.0 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (10.0.1)  
 Requirement already satisfied: pyparsing<3.1,>=2.3.1 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (3.0.9)  
 Requirement already satisfied: python-dateutil>=2.7 in c:\users\urmil\anaconda3\lib\site-packages (from matplotlib->calmap) (2.8.2)  
 Requirement already satisfied: pytz>=2020.1 in c:\users\urmil\anaconda3\lib\site-packages (from pandas->calmap) (2023.3.post1)  
 Requirement already satisfied: tzdata>=2022.1 in c:\users\urmil\anaconda3\lib\site-packages (from pandas->calmap) (2023.3)  
 Requirement already satisfied: six>=1.5 in c:\users\urmil\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->calmap) (1.16.0)

In [179...

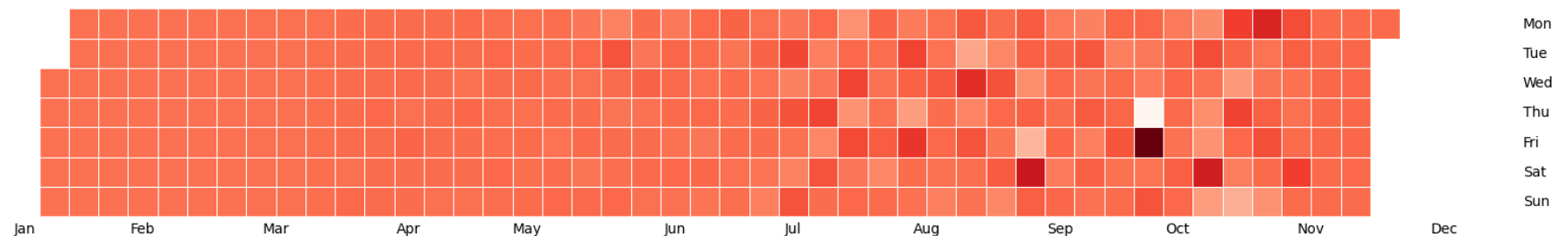
```
import pandas as pd
import calmap
import matplotlib.pyplot as plt

# Assuming `complete_table` is your DataFrame
t = complete_table.groupby('Date')['Deaths'].sum()
t = t.diff()

plt.figure(figsize=(20, 5))
a = calmap.yearplot(t, fillcolor='white', cmap='Reds', linewidth=0.5)

# Add any additional plot customization or labels if needed

plt.show()
```



In [180...

```
HTML('''<div class="flourish-embed flourish-bar-chart-race" data-src="visualisation/13474459"><script src="https://public.flouris
```

Out[180]:

0

500000

1000000

1500000



Confirmed

Deaths

Recovered

Confirmed

```
In [181... complete_table[(complete_table['Country/Region']=="China")].head(5)
```

```
Out[181]:
```

|    | Province/State | Country/Region | Lat     | Long     | Date       | Confirmed | Deaths | Recovered | Check | Active | size     |
|----|----------------|----------------|---------|----------|------------|-----------|--------|-----------|-------|--------|----------|
| 58 | Anhui          | China          | 31.8257 | 117.2264 | 2020-01-22 | 1         | 0      | 0.0       | 1.0   | 1.0    | 1.000000 |
| 59 | Beijing        | China          | 40.1824 | 116.4142 | 2020-01-22 | 14        | 0      | 0.0       | 14.0  | 14.0   | 2.207183 |
| 60 | Chongqing      | China          | 30.0572 | 107.8740 | 2020-01-22 | 6         | 0      | 0.0       | 6.0   | 6.0    | 1.711770 |
| 61 | Fujian         | China          | 26.0789 | 117.9874 | 2020-01-22 | 1         | 0      | 0.0       | 1.0   | 1.0    | 1.000000 |
| 62 | Gansu          | China          | 35.7518 | 104.2861 | 2020-01-22 | 0         | 0      | 0.0       | 0.0   | 0.0    | 0.000000 |

```
In [182... import pandas as pd
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt

# Assuming 'complete_table' is your DataFrame with a datetime index
# Replace 'Deaths' with the actual column name in your DataFrame
time_series = complete_table[(complete_table['Country/Region']=="China") & (complete_table['Province/State']=="Hubei")]['Deaths']

# Ensure your index is a datetime object
time_series.index = pd.to_datetime(time_series.index)

# Decompose the time series
result = seasonal_decompose(time_series, model='additive', period=1) # Specify the period based on the data frequency

# Plot the components
plt.figure(figsize=(12, 8))

plt.subplot(4, 1, 1)
plt.plot(time_series, label='Original')
plt.title('Original Time Series')
plt.legend()

plt.subplot(4, 1, 2)
plt.plot(result.trend, label='Trend')
plt.title('Trend Component')
plt.legend()

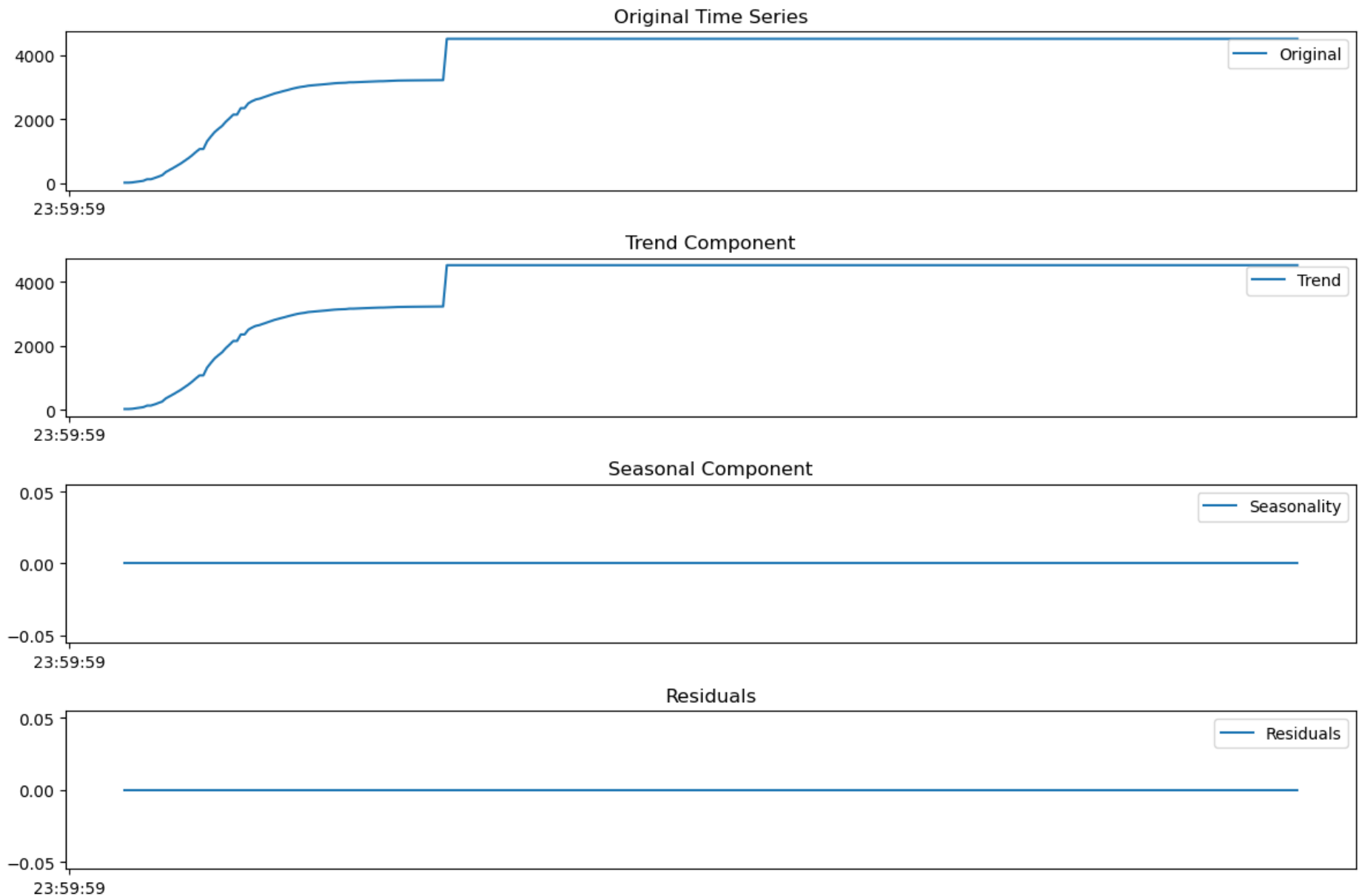
plt.subplot(4, 1, 3)
plt.plot(result.seasonal, label='Seasonality')
```



```
plt.title('Seasonal Component')
plt.legend()

plt.subplot(4, 1, 4)
plt.plot(result.resid, label='Residuals')
plt.title('Residuals')
plt.legend()

plt.tight_layout()
plt.show()
```



```
In [183... import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Assuming 'complete_table' is your DataFrame with a datetime index
```

```
# Replace 'Deaths' with the actual column name in your DataFrame
time_series = complete_table[(complete_table['Country/Region']=="China") & (complete_table['Province/State']=="Hubei")]['Deaths']

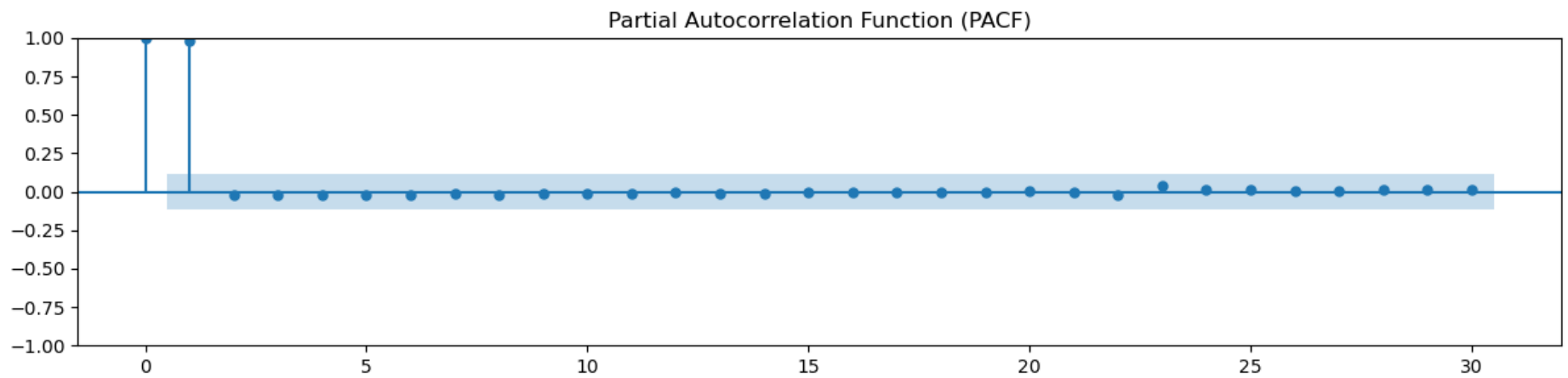
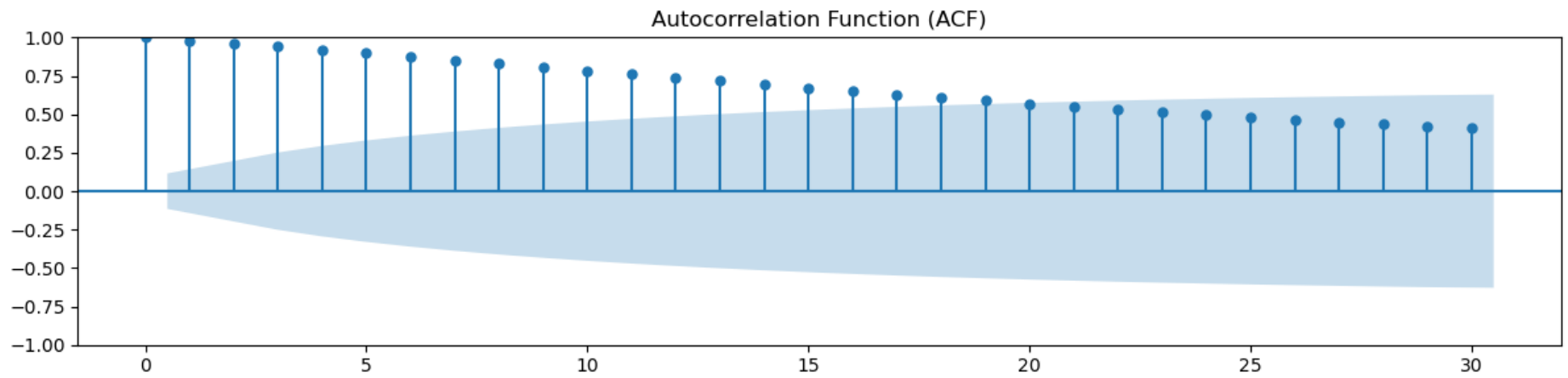
# Ensure your index is a datetime object
time_series.index = pd.to_datetime(time_series.index)

# Plot ACF and PACF
plt.figure(figsize=(12, 6))

plt.subplot(2, 1, 1)
plot_acf(time_series, lags=30, ax=plt.gca()) # Adjust 'lags' based on your data frequency
plt.title('Autocorrelation Function (ACF)')

plt.subplot(2, 1, 2)
plot_pacf(time_series, lags=30, ax=plt.gca()) # Adjust 'lags' based on your data frequency
plt.title('Partial Autocorrelation Function (PACF)')

plt.tight_layout()
plt.show()
```



In [ ]: